

Singleton

PADRÃO CRIACIONAL Uma classe, uma instância e um acesso controlado

Definição. O Singleton garante que uma classe tenha **uma única instância** e fornece um ponto global e controlado para acessá-la. Ele faz sentido quando todos os módulos precisam compartilhar exatamente o mesmo objeto e o mesmo estado.



Analogia: módulos diferentes consultam a mesma central e o mesmo histórico.

Estrutura mínima em Java

```
private static final CentralDeAlertas
    INSTANCIA = new CentralDeAlertas();

private CentralDeAlertas() { }

public static CentralDeAlertas getInstancia() {
    return INSTANCIA;
}
```

Como funciona

- 1. A classe cria e guarda a instância em um campo estático.
- 2. O construtor privado impede novas centrais fora da classe.
- 3. `getInstancia()` sempre devolve o mesmo objeto.

Prova no programa

`centralDaPolicia == centralDosBombeiros` retorna **true**. Os alertas dos três órgãos aparecem no mesmo histórico.

Quando usar

A unicidade faz parte do requisito: configurações da aplicação, registro central de eventos ou gerenciador local de recursos.

Quando evitar

Cada usuário, sessão ou tarefa precisa do próprio estado. Carrinho de compras e usuário autenticado não devem ser Singletons.

Vantagens

- Garante uma instância e centraliza o acesso ao recurso compartilhado.
- Evita objetos duplicados quando a duplicação quebraria o sistema.

Cuidados

- O acesso global pode esconder dependências e dificultar testes isolados.
- Proteger a criação não torna automaticamente o estado interno seguro entre threads.
- A instância é única normalmente por JVM, não entre vários servidores.

Datasheet

Categoria	Criacional
Responsabilidade	Controlar criação e acesso
Método típico	getInstancia()
Sinal de uso	Recurso realmente único

Perguntas rápidas

- Por que não usar new?** Porque ele permitiria criar outra central.
- É uma classe estática?** Não. O Singleton continua sendo um objeto.

REGRA PRÁTICA Use Singleton quando o sistema precisa provar que existe uma única instância compartilhada.